Designing Custom Exceptions

8 min read · Updated 28 Jul 2026 · Topic 30 of 65

Bookmark · Mark done

Why it matters

Exception design is one of the clearest windows into how someone writes production code. Anyone can subclass `RuntimeException`; what interviewers listen for is whether you know when *not* to, what belongs in an exception besides a message, and how exceptions should change shape as they cross layers. It is also directly practical: a well-designed exception turns a support ticket into a two-minute fix.

The concept, simply

A custom exception is worth creating when callers need to **distinguish** this failure from others — to catch it specifically, to map it to a particular HTTP status, or to retry only this case. If nobody will ever catch it by type, a standard exception with a good message is better.

The JDK already covers most situations, and reaching for the standard type is usually the right call:

Situation	Use
Bad argument value	`IllegalArgumentException`
Wrong object state for the call	`IllegalStateException`
Null where it is not allowed	`NullPointerException` (via `Objects.requireNonNull`)
Index out of range	`IndexOutOfBoundsException`

Operation not supported	`UnsupportedOperationException`
A domain concept callers act on	**Your own exception**

How it works

A well-formed exception has four properties:

1. **The right supertype.** Unchecked unless the caller has a real branch (previous topic).
2. **A cause constructor.** Always provide `(String message, Throwable cause)`, or the root cause will be lost the first time someone wraps something.
3. **Structured data, not just a message.** If a handler needs the order id or the decline code, expose it as a field with an accessor — parsing it back out of a message string is a bug waiting to happen.
4. **A message that identifies the instance.** "Order not found" is useless; "Order 88421 not found for customer 3310" is a diagnosis.

Exception hierarchies are how you let callers choose their granularity: a base `OrderException` with `OrderNotFoundException` and `OrderAlreadyShippedException` beneath it means a caller can catch the specific case it handles or the base type for everything else.

Code examples

A well-designed exception:

JAVA

Copy

```java
public class InsufficientFundsException extends RuntimeException {
    private final String accountId;
    private final BigDecimal balance;
    private final BigDecimal requested;

    public InsufficientFundsException(String accountId, BigDecimal balance
        // ✓ A message that identifies the exact instance of the problem.
        super("Account %s has %s but %s was requested".formatted(accountId
```

```
        this.accountId = accountId;
        this.balance = balance;
        this.requested = requested;
    }

    // ✓ Structured accessors — a handler can build a response without
    //   parsing the message string.
    public String accountId() { return accountId; }
    public BigDecimal shortfall() { return requested.subtract(balance); }
}

// The caller can now do something genuinely useful:
catch (InsufficientFundsException e) {
    return Response.status(402)
        .body(Map.of("shortfall", e.shortfall(), "account", e.accountId())
}
```

A hierarchy, so callers pick their own granularity:

JAVA Copy

```
public abstract class OrderException extends RuntimeException {
    private final String orderId;
    protected OrderException(String orderId, String message, Throwable cau
        super(message, cause);            // ✓ cause always available
        this.orderId = orderId;
    }
    public String orderId() { return orderId; }
}

public class OrderNotFoundException extends OrderException { … }
public class OrderAlreadyShippedException extends OrderException { … }

// Specific handling where it matters, blanket handling where it does not:
try {
    orders.cancel(id);
} catch (OrderAlreadyShippedException e) {
    return refundInstead(e.orderId());        // a real, different action
} catch (OrderException e) {
```

Translating at layer boundaries — the pattern that keeps a codebase clean:

JAVA Copy

```java
// Repository: a database concern becomes a domain concern. Callers above
// this line should never see SQLException — they cannot do anything with
public Order findById(String id) {
    try {
        return jdbc.queryForObject(SQL, mapper, id);
    } catch (EmptyResultDataAccessException e) {
        throw new OrderNotFoundException(id, "no order " + id, e);   // ✓
    }
}

// Controller: a domain concern becomes an HTTP concern.
@ExceptionHandler(OrderNotFoundException.class)
public ResponseEntity<?> handle(OrderNotFoundException e) {
    return ResponseEntity.status(404).body(new ErrorBody("ORDER_NOT_FOUND"
}
// Each layer speaks its own vocabulary, and the cause chain preserves the
// whole story for the logs.
```

Validation, where the standard types are better than custom ones:

JAVA Copy

```java
public Order(String id, BigDecimal total, Customer customer) {
    // ✓ Objects.requireNonNull throws NPE with a useful message and reads
    this.customer = Objects.requireNonNull(customer, "customer");
    if (id == null || id.isBlank())
        throw new IllegalArgumentException("id must not be blank");
    if (total.signum() < 0)
        throw new IllegalArgumentException("total must not be negative: "
    this.id = id;
    this.total = total;
}
```

Under the hood

Exceptions are serialisable, because `Throwable` implements `Serializable`. If your exception has fields, they must be serialisable too or serialising it will fail — which matters for anything crossing a remote boundary, and is a reason to keep fields to simple values.

The four-argument `Throwable` constructor (Java 7) lets you disable suppression and stack-trace writability. That is the supported way to build a cheap control-flow exception where profiling shows construction cost matters — for instance a parser that throws to unwind. Reach for it rarely, and never for exceptions a human will read in a log.

Do not put secrets in messages. Exception messages end up in logs, in error responses, and sometimes in front of users. An exception that helpfully includes the connection string or the token that failed has just written a credential to disk. Include identifiers, not payloads.

A note on the modern alternative: for expected, non-exceptional outcomes — "user not found" during a lookup that routinely misses — returning `Optional` or a sealed result type is often better than throwing. Exceptions are for the exceptional; a miss that happens on half of all calls is a normal outcome, and modelling it as a return value makes it impossible for a caller to forget.

Common mistakes

- **Creating an exception nobody catches by type.** If it is never distinguished, a standard exception with a good message is better.
- **Omitting the cause constructor.** Guarantees a lost root cause eventually.
- **Data only in the message.** Handlers end up parsing strings. Expose fields.
- **Vague messages.** "Operation failed" tells nobody anything. Name the entity and the values.

- **Secrets or PII in messages.** They go straight into logs.
- **Exceptions for ordinary outcomes.** Use `Optional` or a result type for the common "not found".
- **A deep hierarchy for its own sake.** Two levels is usually plenty.

Interview lens

> **Q: "When would you create a custom exception?"**
> A: When callers need to distinguish this failure from others — to catch it specifically, map it to a status code, or retry only this case. If nobody will catch it by type, I use a standard exception with a precise message instead. Creating a class per failure mode adds vocabulary the codebase has to carry with no benefit.

> **Q: "What makes a good exception class?"**
> A: The right supertype — unchecked unless the caller has a real branch; a constructor that accepts a cause, so wrapping never loses the root; structured fields with accessors rather than data buried in the message; and a message that identifies the specific instance, with ids and values rather than "operation failed".

> **Q: "Should exceptions cross layer boundaries?"**
> A: No — they should be translated. A repository turns `SQLException` into a domain exception like `OrderNotFoundException`; a controller turns that into an HTTP status. Each layer speaks its own vocabulary, and chaining the cause keeps the full story for the logs. Spring's `DataAccessException` hierarchy exists to do exactly this for persistence.

> **Q: "Exception or `Optional` for 'not found'?"**
> A: Depends whether it is exceptional. If a lookup routinely misses — checking

indicates a broken invariant, such as a foreign key pointing at nothing, an exception is right. The smell is throwing an exception on a path that executes half the time.

Q: "Should a custom exception be checked or unchecked?"

A: Unchecked unless the caller has a concrete alternative action. My test is: can I write the catch block that does something other than log and give up? For a declined payment where the caller offers another method, checked is defensible. For "the order is invalid", unchecked — forcing a `try/catch` there produces empty blocks, not better handling.

Q: "Any risks in exception messages?"

A: Yes — they end up in logs and sometimes in API responses. Including a connection string, a token or personal data writes it somewhere it should not be, and log aggregation spreads it further. I include identifiers and values needed to diagnose, never credentials or payloads. Exceptions are also serialisable, so any field you add must be serialisable too.

Cheat-card

- Create one only when callers **catch it by type**. Otherwise use a standard exception.
- Standard first: `IllegalArgumentException`, `IllegalStateException`, `Objects.requireNonNull`, `UnsupportedOperationException`.
- Always provide a **(message, cause)** constructor.
- **Structured fields + accessors**, not data parsed out of the message.
- Messages name the **entity and values**; never secrets or PII.
- **Translate at layer boundaries**, keeping the cause chain.
- Two levels of hierarchy is usually enough.
- Routine "not found"? Prefer `Optional` over an exception.

1. When is creating a custom exception class justified?

- ◯ When callers need to catch this failure specifically — to handle it differently, map it to a status code, or retry only this case
- ◯ Whenever the message would be longer than one line
- ◯ Only for checked exceptions
- ◯ For every distinct failure in the application

Check answer

2. Which is the most important constructor to provide on a custom exception?

- ◯ One taking a numeric error code
- ◯ One taking (String message, Throwable cause), so wrapping never loses the root cause
- ◯ A copy constructor
- ◯ A no-arg constructor

Check answer

3. A lookup that legitimately misses about half the time. Exception or Optional?

- ◯ An exception, because Optional cannot express failure
- ◯ Always an exception — that is what they are for
- ◯ Optional — an outcome that occurs on half of all calls is not exceptional, and a return value cannot be forgotten by the caller

Either: there is no practical difference

Check answer

Previous
try-with-resources and Suppressed Exceptions

Next
Lambdas and Functional Interfaces

LEARN MORE

ARTICLES & DOCS

- Creating exception classes — The Java Tutorials
  docs.oracle.com
- Chained exceptions — The Java Tutorials
  docs.oracle.com

VIDEOS



Master Exceptions in Java: Try, Catch, Finally, Throw, Throws, try-with-resources & Custom Exception
Engineering Digest

Hand-picked references for this lesson.